AI CAREER COMPANION

FINAL TECHNICAL DOCUMENTATION

MILESTONE 4.4

Validation, Optimization & Release Readiness

Prepared for: Infosys Internship Project

Project: AI Career Companion

Milestone: Milestone 4.4

Author: Jeswin Jarald

Table of Contents

1. Executive Summary 3
2. Problem Statement 3
3. Objectives 4
4. System Architecture 5
4.1 Logical flow 5
4.2 Optional LLM path (separate from the core flow) 5
4.3 Why this is not a "vector-only RAG chatbot" 6
5. Technology Stack 6
6. Data and Knowledge Base 6
7. Resume Processing Pipeline 6
8. Semantic Retrieval 7
9. Matching Engine 7
10. Skill Gap Analysis 8
11. Resume Customization 8
12. Interview Preparation 8
13. Conversational Assistant 9
14. Application Tracker 9
15. M1 Implementation 9
16. M2 Implementation 10
17. M3 Implementation 10
18. M4.1 Implementation 10
19. M4.2 Evaluation 10
20. M4.3 Optimization 12
21. Testing Strategy 14
22. Security and User Isolation 14
23. Performance 15
24. Final Evaluation Results 16
25. Demo Workflow 16
26. Known Limitations 17
27. Future Enhancements 18
28. Setup / Run Instructions 18
29. API Documentation 19
Database 20
30. Repository / Git Hygiene 20

Infosys internship project. This is the canonical end-of-project document (Milestone 4.4). It summarises the whole system and links to the per-milestone documents and evaluation artifacts for detail instead of repeating them. Every metric quoted here is taken from a tracked evaluation artifact or from the M4.4 verification runs recorded in §21 and §30; anything that was not measured is stated as such.

Status of this document: describes the code on branch `develop` as committed in M4.4. The project is a local development / evaluation implementation. It is **not deployed**: there is no production server, cloud hosting, container image, CI/CD pipeline or production monitoring in this repository.

1. Executive Summary

AI Career Companion is a web application that helps a student turn their resume into concrete career actions. A student registers, uploads a PDF or DOCX resume, and the system extracts structured candidate information from it. That candidate context is then used to:

- retrieve relevant opportunities from a 320-record career-opportunity knowledge base using sentence-transformer embeddings and a FAISS index;
- score each opportunity against the candidate with a **deterministic, weighted, explainable** matching function;
- produce an evidence-based skill-gap analysis for a chosen opportunity;
- generate a tailored resume summary / project ordering and a cover letter whose claims are validated against the candidate's real evidence;
- generate interview preparation tied to the candidate's projects, the job's requirements and the skill gaps;
- answer follow-up questions in a conversational assistant that routes to the same services;
- track applications (status, user-entered deadlines, interview dates, notes, linked generated materials) and surface reminders.

The system is a **deterministic + optional-LLM hybrid**. The core application workflows operate without an LLM (`LLM_PROVIDER=none`, the default), using deterministic logic and grounded fallbacks; general-purpose career questions remain limited in deterministic mode (limitation CV06, §26). When an OpenAI-compatible LLM is configured, it is used only to *rewrite* already-grounded content, its JSON output is schema- and evidence-validated, and any failure falls back to the deterministic result.

Headline evaluation results (offline, deterministic pipeline, M4.3 final evaluation over 26 labelled retrieval queries, 12 off-topic queries, 5 synthetic candidates / 8 candidate–job pairs and 7 scripted conversations / 16 turns):

Measure	M4.2 baseline	M4.3 final
Retrieval Hit@1 / MRR@10	0.885 / 0.942	1.000 / 1.000
Grounding violations (structured checks)	0	0
Fault injections detected by the evaluator	9/9	10/10
Conversation turns with desired behaviour	12/16	15/16
Backend test suite	374 passed, 6 xfailed	464 passed, 1 xfailed

These describe the evaluation fixtures, not the population of real resumes or jobs (see §26).

2. Problem Statement

Students applying for internships and entry-level roles typically:

- do not know which openings genuinely fit their current skills;

- cannot see precisely which requirements they are missing for a specific role;
- write generic resumes and cover letters, or — when using generative tools — risk including skills and experience they do not have;
- prepare for interviews without connecting questions to their own projects and gaps;
- lose track of where they applied, deadlines and interview dates.

General-purpose chatbots address some of this, but they are not grounded in the student's actual resume or in a fixed opportunity catalogue, and they can fabricate qualifications.

3. Objectives

1. Extract structured, inspectable candidate context from a real resume (no OCR, no LLM required).
2. Retrieve relevant opportunities semantically from a curated, validated knowledge base.
3. Score candidate–job fit with transparent, deterministic, weighted logic.
4. Make every generated statement traceable to candidate evidence; never claim a skill the candidate has not shown.
5. Keep the optional LLM strictly subordinate: grounded input, validated JSON output, deterministic fallback.
6. Persist each user's data under strict ownership; prevent cross-user access.
7. Measure the complete workflow (M4.2), improve it under controlled experiments (M4.3), and document the result honestly (M4.4).

4. System Architecture

4.1 Logical flow

```
User (browser)
   │
   ▼
React 19 + Vite + TypeScript SPA  (src/)
   │  fetch(), credentials: include  →  VITE_API_BASE_URL (default http://localhost:8000)
   ▼
FastAPI application  (backend/app/main.py, 11 routers)
   │
   ▼
Authentication (HttpOnly session cookie → hashed session token lookup)
and ownership checks (every user-data query filtered by the current user)
   │
   ▼
Profile + Resume pipeline
   upload/validate → text extraction → section detection → structured resume
   │
   ▼
Candidate structured context  (profile + structured resume, persisted)
   │
   ▼
┌──────────────────────────────────────────────────────────────────┐
│ Opportunity retrieval                                            │
│ query text → all-MiniLM-L6-v2 (384-d, normalised)                │
│ → FAISS IndexFlatIP over 960 chunk vectors (index v1.2)          │
│ → chunk scores aggregated per job → query-confidence flag        │
└──────────────────────────────────────────────────────────────────┘
   │  retrieved job IDs → canonical structured job records
   ▼
Matching (deterministic weighted score) / Skill Gap (evidence-based readiness)
/ Customization (resume + cover letter, grounding validator)
   │
   ▼
Interview Preparation / Conversational Assistant
   │
   ▼
Application Tracker (statuses, dates, notes, linked materials, reminders)
   │
   ▼
SQLAlchemy 2 ORM → SQLite (local file, gitignored)
```

4.2 Optional LLM path (separate from the core flow)

```
Deterministic result (already grounded)
   │
   ├─ LLM_PROVIDER=none (default) ──────────────► deterministic result shown
   │                                              ("Grounded Fallback" badge)
   │
   └─ LLM_PROVIDER=openai_compatible
        build grounded input: evidence items with IDs, allowed claims,
        unsupported terms, strict JSON output contract
           │
           ▼
        POST {LLM_BASE_URL}/chat/completions  (response_format = json_object)
           │
           ▼
        parse JSON → Pydantic schema validation → evidence-ID / unsupported-keyword /
        fabrication validation  ── fails? one repair attempt ── still fails?
           │                                                        │
           ▼                                                        ▼
        LLM-assisted result                          deterministic fallback, with a
                                                     recorded fallback_reason
```

The LLM is used by customization (M3.2), interview preparation (M3.3) and assistant answer synthesis (M3.4). It is never used for retrieval, scoring, skill-gap decisions, the tracker, or authentication. No vendor SDK is used — the provider is a small `httpx` client against any OpenAI-chat-completions-compatible endpoint (`backend/app/services/llm_provider.py`).

4.3 Why this is not a "vector-only RAG chatbot"

Retrieval only selects *which* canonical job records are relevant. All downstream reasoning (scores, gaps, keyword support, question topics) operates on the **structured** job record and the **structured** candidate context with deterministic code. The vector store never supplies free text to an LLM as the sole source of truth; generated text is checked against structured evidence (see §11 and §12).

5. Technology Stack

Layer	Technology (versions from `package.json` / `backend/requirements.txt`)
Frontend	React 19, TypeScript ~5.8, Vite 6, ESLint 9 (typescript-eslint, react-hooks)
Frontend styling	Plain CSS (`src/App.css`, `src/index.css`, `src/profile.css`, `src/components/applications/applications.css`)
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2 / pydantic-settings
Persistence	SQLAlchemy 2.x ORM, SQLite (default `backend/data/ai_career_companion.db`)
Resume parsing	`pypdf` (PDF), `python-docx` (DOCX); no OCR
Embeddings	`sentence-transformers` with `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions)
Vector search	`faiss-cpu`, exact `IndexFlatIP` on L2-normalised vectors (cosine similarity)
Auth	`pwdlib[argon2]` password hashing; opaque session tokens (SHA-256 hashed at rest)
Export	`reportlab` (PDF) and `python-docx` (DOCX) for customization export
Optional LLM	Any OpenAI-compatible `/chat/completions` endpoint via `httpx`
Tests	`pytest`, FastAPI `TestClient` (`httpx`)

There are no schema-migration tools (no Alembic). Tables are created with `Base.metadata.create_all`, plus one small in-place `ALTER TABLE` helper for older local SQLite files that predate some `candidate_profiles` columns (`backend/app/core/database.py`).

6. Data and Knowledge Base

- **Canonical dataset:** `backend/data/internships/career_opportunities_320.json` (CSV mirror `career_opportunities_320.csv`, JSON Schema `job_posting_schema.json`, validation summary `validation_report.json`). Details: `backend/data/internships/README.md`.
- **Size and coverage (re-validated in M4.4):** 320 records, 0 duplicate IDs, 0 duplicate content, 15 domains with 21–22 records each: Cloud Computing, Cybersecurity, Data Analytics, Data Science, Database / SQL, DevOps, Frontend Development, Full Stack Development, Generative AI / NLP, Java Backend, Machine Learning, Mobile Development, Python Backend, Software Development, Software Testing / QA.
- **Opportunity types** (`employment_type`): internships, entry-level jobs, graduate programs, trainee and apprenticeship roles (the M3 generalization — `docs/CAREER_OPPORTUNITY_GENERALIZATION.md`).
- **Nature of the data:** the catalogue is a curated, project-authored knowledge base (company names are not real employers). There is no live job-board integration, no application URLs, and no scraped postings. Within one domain, postings share the same required/preferred skill lists and differ by title, company, location, type and description — this matters for the evaluation label policy (§19).
- **Integrity:** SHA-1 of the JSON is `d9401f9085e7edd9ee9dd8725f344e38ef07aa30`, unchanged from M4.3 through M4.4.

7. Resume Processing Pipeline

Implemented in M1 (detail: `docs/MILESTONE_1.md`).

1. **Upload** — `POST /api/profiles/{profile_id}/resumes` (multipart `file`). Content is validated, not just the extension: PDF signature/structure via `pypdf`, DOCX ZIP and required XML parts via the standard library. 10 MiB limit. Files are stored under `RESUME_STORAGE_DIR` (default `backend/data/resumes/`, gitignored) with generated names; the internal path is never returned.
2. **Text extraction** — `POST /api/resumes/{id}/extract-text`. PDF via `pypdf`; DOCX paragraphs and table rows in document order. Image-only PDFs fail explicitly (no OCR).
3. **Section detection** — `POST /api/resumes/{id}/detect-sections`. Deterministic, alias-based headings (summary, skills, education, experience, internships, projects, certifications, …).
4. **Structured resume** — `POST /api/resumes/{id}/structure`. Deterministic extraction of skills, education, experience, projects, certifications and summary into JSON.
5. **Candidate context** — `POST /api/profiles/{profile_id}/candidate-context?resume_id=` combines the saved career profile with the structured resume; this is the input to all later services.

The frontend runs these steps in sequence when the user clicks *Process resume*. No LLM is involved in parsing.

8. Semantic Retrieval

Implemented in M2.2, generalized in M3, optimized in M4.3 (detail: `docs/MILESTONE_2_2.md`).

- **Chunking** (`job_chunking.py`): each posting becomes three chunks — `overview`, `requirements`, `responsibilities` — with stable IDs such as `JOB-0001::requirements`. Since index **v1.2** (M4.3 experiment E8-V3) the overview chunk starts with `"<title> (<employment type>). "`; the other chunks are unchanged.
- **Embedding** (`embedding_service.py`): `all-MiniLM-L6-v2`, 384-d, L2-normalised. The model is loaded from the local Hugging Face cache first; it is downloaded only on a cache miss and only if `EMBEDDING_ALLOW_DOWNLOAD=true` (default).
- **Index** (`job_vector_store.py`): exact `faiss.IndexFlatIP`, 960 vectors (320 × 3); inner product on normalised vectors equals cosine similarity. Metadata (`internship_jobs_metadata.json`) records `index_version: "1.2"`, model name, dimension 384 and 960 chunk records. The index lives in `backend/data/vector_store/`, which is **gitignored** and rebuilt with `scripts/build_job_vector_index.py`.
- **Search** (`job_search_service.py`): all chunks are searched; per-job score = best chunk score + 0.05 × sum of that job's other chunk scores; top-k jobs are returned.
- **Query confidence** (`retrieval_confidence.py`, M4.3 E6): each result carries `query_confidence` = `confident` or `unsupported_area`, computed from how many of the query's content words occur in a vocabulary derived at runtime from catalogue titles, domains and skills (no hard-coded domain list). Results are **never suppressed**; the Careers page shows a notice and the assistant says the area is not covered.
- **Caching** (M4.3 E10): the validated dataset and loaded vector store are cached in-process, keyed by file path, mtime and size (plus model name for the index). No user data is cached.

9. Matching Engine

Implemented in M2.3, refined in M4.3 (detail: `docs/MILESTONE_2_3.md`). **The match score is computed by deterministic code, not by an AI model.**

1. The candidate's normalised terms (resume + profile) form the retrieval query (or the user's explicit query, e.g. an assistant discovery request).
2. Semantic retrieval returns the top-k candidate jobs (default 10, max 20).
3. Each retrieved job is scored component by component (`job_matching.py`):

Component	Weight
Required skills (exact/alias match)	0.35
Preferred skills (exact match; related technology = 0.5 credit, M4.3 E3e)	0.15
Experience	0.15
Project relevance	0.15
Education (shared `education_assessment`: met 1.0 / partial 0.5 / not met 0.0)	0.10
Qualifications	0.10

Score = Σ(weight × component score) / Σ(weights of components that apply) × 100. Components that cannot be assessed are excluded rather than scored as zero.

4. Each result includes matched / missing required and preferred skills, the skills that are only self-reported in the profile (M4.3 E7), and a plain-language reasoning string.

Related-technology credit applies **only to preferred skills**: a hard requirement needs direct evidence. The related-skill table (`skill_relationships.py`) is shared by matching, skill gap and customization.

10. Skill Gap Analysis

Implemented in M3.1 (detail: `docs/MILESTONE_3_1.md`).

- Compares the candidate against one selected job requirement by requirement: required skills, preferred skills, experience, education, qualifications.
- Each item is classified (matched / partial / missing) with severity, quoted evidence from the resume or profile, and a recommendation. Skills present only in the profile are described as **self-reported** with lower confidence (0.70 vs 0.95, M4.3 E7). Learning-only mentions never count as demonstrated.
- **Readiness** is a separate deterministic weighted score: required 0.40, preferred 0.15, experience 0.20, education 0.10, qualifications 0.15. It is intentionally not the same number as the match score.
- Education uses the same `education_assessment` as matching (M4.3 E4), so the two services no longer disagree on the same pair.

11. Resume Customization

Implemented in M3.2 (detail: `docs/MILESTONE_3_2.md`).

- Produces, per resume × job: keyword alignment (supported / partially supported / unsupported), a tailored professional summary, reordered skills, ranked projects with tailored skill lists, and a cover letter. Each generated item lists its supporting evidence IDs.
- **Grounding validator** (`customization_validator.py`): removes sentences that introduce unsupported skills, invented metrics, leadership claims or years of experience. Naming the target job title and company is not treated as a skill claim (M4.3 E5); the metric / leadership / years checks still see the full text.
- Results are versioned, editable and exportable as PDF / DOCX, and can be linked to a tracker application.
- The UI labels each result **"Grounded Fallback"** (deterministic) or LLM-assisted, and shows whether the grounding check passed.

12. Interview Preparation

Implemented in M3.3 (detail: `docs/MILESTONE_3_3.md`).

- Generates questions in six categories (technical, resume, project, role, HR, skill-gap) with difficulty levels, plus prioritised revision topics, from the candidate's real projects, the job's requirements and the skill-gap result. Each question carries its evidence/source.
- Mock-answer practice: `POST …/interview-preparations/{id}/mock-answer` returns structured feedback (strengths, improvements, missing points, suggested structure).
- Same LLM contract as customization: optional rewrite, validated, deterministic fallback.

13. Conversational Assistant

Implemented in M3.4, improved in M4.3 (detail: `docs/MILESTONE_3_4.md`).

- **Deterministic intent routing** (`assistant_intent.py`): job discovery, match explanation, skill gap, learning guidance, customization, interview prep, job comparison, next best action, general chat.
- **Context resolution** (`assistant_context.py`): the active resume and the job in focus are carried across turns; switching jobs does not leak the previous job.
- **Handlers** call the same matching / skill-gap / customization / interview services — the assistant does not compute its own scores.
- M4.3 changes: discovery retrieves the *requested* area while scoring against the resume (E1); "compare me with JOB-X" is routed as a fit explanation, learning-priority follow-ups and free-form discovery wording are recognised (E2a).
- General career questions (e.g. salary negotiation) receive a fixed capability message in deterministic mode — see limitation CV06 (§26).
- Conversations are persisted per user and can be listed, reopened and deleted.

14. Application Tracker

Implemented in M4.1 (detail: `docs/MILESTONE_4.md`).

- Track an opportunity from Careers (`source = dataset`, server-side snapshot of company, title, type, location, work mode and description) or a manual/external application (`source = manual`).
- 10 statuses: saved, planning, applied, under_review, shortlisted, interview_scheduled, interview_completed, offer, rejected, withdrawn.
- **User-entered** dates: application date, deadline, follow-up date, interview date-time (stored in UTC) and interview status; free-text notes. Deadlines are never inferred — the catalogue has no deadlines.
- Link the generated customization and interview preparation for the same job.
- Summary counts and a reminders list (deadline within 7 days for not-yet-applied items, upcoming interviews, due follow-ups, and applied/under-review items with no change for 14 days). Purely database queries and date rules — no embeddings or LLM.
- The dashboard's "Application activity" block reads the same summary endpoint.

15. M1 Implementation

Candidate-understanding foundation (`docs/MILESTONE_1.md`). That document was written when authentication was still deferred; authentication and per-user ownership were added in the M1/M2 integration commit (`81e7035`).

- Account registration / login / logout / current user with session cookies; a candidate profile is created per user at registration.
- Career profile CRUD (education, degree, specialization, experience level, interests, target roles, skills, goals).

- Resume upload with content validation, PDF/DOCX text extraction, section detection, deterministic structured extraction, and persisted candidate context.
- Dashboard driven by the actual resume lifecycle: a new account shows "No resume analyzed yet. No skills extracted yet. No job matches yet." until a resume is processed.
- Ownership: every profile/resume query is filtered by the authenticated user.

16. M2 Implementation

- **M2.1** opportunity knowledge base (originally 180 internship records; superseded by the 320-record dataset in M3).
- **M2.2** semantic retrieval: chunking, local embeddings, FAISS exact index, job-level aggregation (`docs/MILESTONE_2_2.md`).
- **M2.3** job–resume matching: deterministic weighted scoring with explanations (`docs/MILESTONE_2_3.md`).
- **M2.4** evaluation over 8 student profiles on the 180-record dataset; historical artifacts preserved unchanged in `backend/data/evaluation/results/` (`M2_4_EVALUATION_REPORT.md`, `m2_4_results.json`).
- RAG-style context: retrieved job IDs resolve to structured records that feed matching, not free-text prompts.

17. M3 Implementation

- **M3.1** Skill Gap Analysis agent (evidence-based, deterministic readiness).
- **M3.2** Resume & cover-letter customization with grounding validator and optional LLM rewrite layer with deterministic fallback.
- **M3.3** Interview preparation agent with mock-answer feedback.
- **M3.4** Conversational career assistant with persisted conversations and intent routing.
- **Career-opportunity generalization** (end of M3): canonical dataset expanded to 320 records across 15 domains and five opportunity types; index rebuilt (960 chunks/vectors); terminology generalized from "internships" to "opportunities" (`docs/CAREER_OPPORTUNITY_GENERALIZATION.md`).

18. M4.1 Implementation

Application tracker — backend model `applications`, schema, service, router `/api/applications`, 7 endpoints; frontend `src/components/applications/*` and `src/services/applicationService.ts`; "Add to Tracker" on job details, customization and interview prep pages; dashboard activity block. One canonical job can be tracked once per user (unique `user_id + job_id`); manual rows are unlimited. Linked artifacts must belong to the same user and job. Tests: `test_application_service.py`, `test_applications_api.py`. Full description: `docs/MILESTONE_4.md`.

19. M4.2 Evaluation

A measurement framework for the full workflow (`backend/app/services/m4_evaluation.py`, `backend/scripts/run_m4_evaluation.py`, cases and results in `backend/data/evaluation/m4/`, described in its `README.md`). It runs offline, forces `LLM_PROVIDER=none`, and wraps the run in a null LLM so a live key is never used.

Section	Size
Positive retrieval queries	26 (136 relevant + 612 acceptable job labels, rule-derived)
Negative / off-topic queries	12 (unrelated, nonsense, out-of-coverage)
Synthetic candidates / candidate–job pairs	5 / 8
Matching scenarios (required-strong, preferred-strong, semantic-not-exact, unsuitable)	4
Conversations / turns	7 / 16
Job-discovery probes	3
Grounding structured checks	921 (baseline), 926 (final)

It measures retrieval (Hit / Precision / Recall / nDCG @1,3,5,10, MRR@10), off-topic behaviour, matching properties, matching ↔ skill-gap consistency, grounding (absent skills never claimed, evidence quotes exist, no invented projects/experience/education), the service hand-off chain (same resume/job through match → gap → customization → interview prep), conversation behaviour and cross-user isolation (E2E tests).

Frozen baseline: `m4_2_baseline_results.json` and `M4_2_BASELINE_REPORT.md` are never overwritten. Weaknesses found by the baseline:

- Hit@1 0.885 (RP12, RP16, RP25 had the relevant job at rank 2).
- No off-topic handling: every off-topic query returned confident-looking results (0/12 flagged); score ranges of genuine and off-topic queries nearly touched (gap 0.0004).
- Matching: semantic-but-not-exact candidate (C) scored the same as unsuitable (D) — 21.1 vs 21.1; A−B margin only 2.8.
- 2 education disagreements between matching and skill gap.
- 4 needs-review items (profile-only skills shown as demonstrated; one validator removal of a legitimate cover-letter opening).
- Conversation 12/16; job discovery returned identical jobs for different requested areas (0/3).
- General career question (CV06) answered with a capability message.

The test suite encodes these as correctness invariants, regression floors, and strict `xfail` tests for each known weakness (`tests/test_m4_2_baseline.py`).

20. M4.3 Optimization

Each change was measured alone against the frozen M4.2 baseline with the same evaluation, then accepted or reverted. Full records (hypothesis, change, baseline, result, regressions, decision): `backend/data/evaluation/m4/results/m4_3_experiments.json`; summary: `M4_3_OPTIMIZATION_REPORT.md`.

ID	Hypothesis	Change	Result	Decision and reason
E1	Discovery ignores the requested area (always retrieves with resume terms).	Discovery wording stripped; remaining area words drive retrieval; scoring stays resume-based.	Discovery expected-domain hit 0.0 → 1.0; results differ per request.	**ACCEPT** — no regressions.
E2a	"Compare me with JOB-X", learning-priority follow-ups and free-form discovery are mis-routed.	Deterministic regex rules in intent classification (whole-word, so "machine learning" does not fire).	Conversation turns 12/16 → 15/16.	**ACCEPT** — existing intent tests unchanged.
E2b	General questions get a fixed capability message.	None implemented.	CV06 still fails.	**REJECT / deferred** — needs the LLM path; not measurable offline; changing fixed text to pass would be overfitting.
E3a	Matching is exact-term only.	Shared bidirectional related-skill table; 0.5 credit for related required **and** preferred skills.	A 56.5, **B 61.8**, C 35.8; 1 new grounding violation.	**REJECT** — B (1/4 required) overtook A (3/4 required).
E3b	Gating preferred credit restores required-skill importance.	E3a + preferred score × required score.	A 56.1, B 53.4; P001 and P005 fell below expected scores.	**REJECT** — M2 profile regressions.
E3c	Raising the required weight widens A–B.	E3b + required 0.45 / preferred 0.10.	A–B 7.3; P005 regressed.	**REJECT** — target not met and regression.
E3e	Related technologies are valid partial evidence only for nice-to-have skills.	Shared table; 0.5 credit for related **preferred** skills only (kept in "missing", explained); fixed `node.js`/`nodejs` normalisation defect.	A 56.5, B 52.0, C 26.1, D 21.1; M2 behaviour identical.	**ACCEPT** — C > D now holds; A–B ≥ 10 and C–D ≥ 10 targets **not** met.
E4	Matching and skill gap assess education differently.	One shared `education_assessment` (met / partial / not_met / unknown).	Consistency findings 2 → 0.	**ACCEPT** — no regressions.
E5	The validator removed a cover-letter opening because the job title contains a skill the student lacks.	Job title and company passed as allowed phrases (unsupported-keyword check only), in final and LLM-layer validators and in the evaluator; 10th fault injection added.	Review items 1 → 0; faults 10/10 detected; violations 0.	**ACCEPT**.
E6	Score overlap makes a similarity threshold unsafe; query-term coverage separates off-topic queries.	Variants: A absolute threshold, B top1−top5 margin, C query-term coverage, E hybrid.	C (<0.5): 11/12 off-topic flagged, 0/26 genuine flagged. A flagged genuine queries at useful thresholds; B flagged 7–13 genuine; E no better than C.	**ACCEPT C as a flag only**; A, B, E **REJECT**.
E7	Profile-only skills are presented as demonstrated.	Described as self-reported, confidence 0.70; matching records `self_reported_skills`.	Review items 3 → 0; scores unchanged.	**ACCEPT**.

ID	Hypothesis	Change	Result	Decision and reason
E8-V1	Title/type distinguish postings inside a domain.	Extra 4th title/type chunk (1280 vectors), in memory only.	Hit@1 1.0 but Precision@5 0.6385 and Recall@5 0.8577 below baseline; index +33 %.	**REJECT**.
E8-V2	Same.	Title/type prefix on all three chunks; index v1.1 built.	Hit@1 1.0, MRR 1.0; but an integration test's ML profile lost its ML job from the top 5.	**REJECT (reverted)**.
E8-V3	Adding title/type only to the overview chunk avoids diluting skill chunks.	Overview chunk prefixed; `INDEX_VERSION` 1.2; canonical index rebuilt (960 vectors).	Hit@1 1.0, MRR 1.0, P@5 0.6615, R@5 0.8831, nDCG@5 0.9693; integration profiles 4/4; 5/5 held-out title/type queries at rank 1.	**ACCEPT** — no regressions.
E9	Hub checks on every cold model load make startup slow and can hang offline.	Load with `local_files_only=True` first; download only on cache miss if allowed.	Cold load 12.1–12.3 s → 7.3 s; suite no longer hangs without `HF_HUB_OFFLINE`.	**ACCEPT**.
E10	Each request re-reads and re-validates the dataset and index.	Keyed in-process caches; invalidated on file change.	e.g. `search_jobs` 46.71 → 9.44 ms median (see §23).	**ACCEPT**.
T1	Token usage is not recorded.	Provider records reported `usage`; nothing estimated.	Verified with fake HTTP responses only.	**ACCEPT** (instrumentation).
P1	Prompts may need revision.	Prompt review; no text changed (not measurable without live calls).	One prompt/validator conflict fixed under E5.	**NO PROMPT CHANGE**.

Strict xfails went from 6 to 1; the remaining one is CV06.

21. Testing Strategy

- **Unit and service tests** for parsing, section detection, structuring, dataset validation, chunking, vector store, search, matching, skill gap, customization and its validator, interview prep, assistant intent/context/handlers, LLM provider (fake HTTP), application tracker.
- **API tests** with FastAPI `TestClient` against per-test temporary SQLite databases and resume folders (pytest `tmp_path` fixtures), including authentication and cross-user 404/401 behaviour.
- **No live LLM in tests**: `tests/conftest.py` is a suite-wide autouse guard that patches every LLM-provider binding to `NullLLMProvider`, so a real key in `backend/.env` can never be used by the suite; LLM behaviour is tested with explicit fake providers.
- **Evaluation-backed tests**: `test_m4_2_evaluation_framework.py` re-derives every retrieval label from its rule (a silent dataset change fails the suite); `test_m4_2_baseline.py` holds invariants, regression floors and strict xfails; `test_m4_2_e2e_workflow.py` runs register → profile → upload → extract → sections → structure → candidate context → job matches → skill gap → customization → interview prep → tracker (create, duplicate 409, link, dates, summary, reminders, status change) → logout/login, then checks that a second user cannot read or link any of it (the assistant is covered by its own test modules and the M4 conversation evaluation); `test_m4_3_optimizations.py` covers each accepted experiment.
- **Frontend**: TypeScript strict type-check, ESLint, and production build. There is no automated frontend test framework in this project; the UI is validated manually (§25).

M4.4 verification run (2026-10-05, Windows 11, Python 3.12.3):

Gate	Command	Result
Backend suite	`cd backend && ..\.venv\Scripts\python.exe -m pytest -q`	**464 passed, 0 failed, 1 xfailed, 2 warnings** in 39.2 s (warnings: Starlette `httpx` TestClient deprecation and `anyio.abc.BlockingPortal` alias deprecation, both third-party)
Dataset validation	`python scripts/validate_job_dataset.py`	**PASS** — 320 records, 15 domains, 0 duplicates
Index validation	load vector store	IndexFlatIP, ntotal 960, d 384, metadata `index_version` 1.2
M4 evaluation (M4.2 framework, M4.3 state)	`python scripts/run_m4_evaluation.py --output-dir <scratch>`	All metrics identical to the tracked `M4_3_OPTIMIZED_EVALUATION.md` (only timestamp/timings differ; runtime 2.5 s)
TypeScript	`npx tsc -b`	pass (0 errors)
ESLint	`npm run lint`	0 errors, 1 warning (`react-hooks/exhaustive-deps` in `ProfileView`; the same line exists in the pre-M4 commit — not introduced in M4)
Production build	`npm run build`	pass (48 modules; JS 316.66 kB / 88.81 kB gzip)
API startup	`uvicorn app.main:app`	startup complete; `GET /health` → `{"status":"ok"}`; `/docs` and `/openapi.json` → 200
Manual demo	§25 against a fresh scratch database	pass (see §25)

22. Security and User Isolation

Implemented mechanisms (verified in code and in the M4.4 manual run):

- **Passwords** hashed with `pwdlib` recommended hasher — stored values are Argon2id (`$argon2id$…`); plaintext is never stored.

- **Sessions**: a random `secrets.token_urlsafe(48)` token is sent as cookie `ai_career_session` with `HttpOnly`, `SameSite=Lax`, 14-day max-age. Only its SHA-256 hash is stored (`auth_sessions.token_hash`). Logout deletes the session row. The cookie is set with `secure=False` because the project runs over local HTTP.
- **Authorization**: every user-data router (profiles, resumes, candidate context, skill gap, customization, interview prep, assistant, applications) depends on `require_current_user`; queries are filtered by the current user and foreign objects return **404** (not 403) so their existence is not revealed. Unauthenticated requests get **401**. `/health` and the read-only opportunity catalogue (`/api/jobs/*`) are public.
- **Cross-user isolation**: covered by API/E2E tests; additionally probed manually in M4.4 — a second user received 404 for the first user's application (GET/PATCH/DELETE), resume, structured resume, profile, candidate context, customizations, skill gap and conversation; their own application list was empty.
- **Uploads**: content-type validated by signature/structure, size-limited, stored under generated names outside any served path.
- **CORS**: credentialed; the configured `FRONTEND_URL` is allowed exactly, and any `localhost`/`127.0.0.1` port is allowed only when `APP_ENV=development`.
- **Secrets**: `LLM_API_KEY` is read from the environment / local `backend/.env` only and never logged (startup logs only whether a key is present). `.env` files are gitignored; only `.env.example` placeholders are tracked.
- **Unhandled errors** return a generic `{"detail": "Internal server error"}`.

Not implemented (do not assume): OAuth/SSO, JWT, CSRF tokens (mitigated only by `SameSite=Lax`), rate limiting / login throttling, account lockout, email verification, password reset, encryption at rest, HTTPS configuration, audit logging, or any penetration testing.

23. Performance

All figures are **medians of repeated warm calls on the single development machine used for M4.3**, deterministic pipeline, no LLM. They are not production latency guarantees.

Stage	Before M4.3	After M4.3
Dataset load + validation	17.87 ms	0.24 ms
Vector store load	4.94 ms	0.14 ms
Query embedding	7.67 ms	5.27 ms
FAISS search (960 vectors)	0.39 ms	0.09 ms
`search_jobs` total	46.71 ms	9.44 ms
`match_jobs_for_resume` (top 10)	78.05 ms	16.31 ms
Skill gap	27.61 ms	3.07 ms
Customization (deterministic)	20.04 ms	4.57 ms
Cold embedding-model load (network allowed)	12.1–12.3 s	7.3 s
M4 evaluation runtime	5.4–5.5 s	2.2 s (2.5 s in the M4.4 re-run)
Full backend suite without `HF_HUB_OFFLINE`	hung > 20 min	30.5 s (39.2 s in M4.4 run)

Embedding and FAISS code were not changed; their small differences are run-to-run variation. LLM latency and token usage were **not measured** (live LLM calls are disabled in the evaluation).

24. Final Evaluation Results

Source: `backend/data/evaluation/m4/results/M4_3_OPTIMIZED_EVALUATION.md` and `m4_3_optimized_results.json` (reproduced identically in M4.4). The report generator (`render_markdown` in `m4_evaluation.py`, called by `run_m4_evaluation.py`) titles the report "Milestone 4.3 Optimized Evaluation Report", and its configuration line records that there is no score threshold and that each result carries the query-confidence flag. The report was regenerated after that correction; every measured value is unchanged (only the timestamp and timings differ). The frozen M4.2 baseline report keeps its original title.

Retrieval (26 labelled queries)

Metric	M4.2 baseline	M4.3 final
Hit@1 / @3 / @5 / @10	0.885 / 1.000 / 1.000 / 1.000	1.000 / 1.000 / 1.000 / 1.000
Precision@1 / @3 / @5 / @10	0.885 / 0.833 / 0.654 / 0.396	1.000 / 0.846 / 0.661 / 0.404
Recall@1 / @3 / @5 / @10	0.314 / 0.738 / 0.874 / 0.926	0.404 / 0.751 / 0.883 / 0.936
nDCG@1 / @3 / @5 / @10	0.923 / 0.968 / 0.961 / 0.938	1.000 / 0.984 / 0.969 / 0.938
MRR@10	0.942	1.000
Top-1 domain accuracy	1.000	1.000

Recall@k is bounded by k / label count (many queries have more relevant jobs than k); see the metric definitions in the evaluation report.

Other sections

Measure	M4.2 baseline	M4.3 final
Off-topic queries flagged	0/12	11/12 (miss: "UI UX design internship with Figma")
Genuine queries flagged	0/26	0/26
Held-out confidence check	—	off-topic 10/10 flagged; genuine 1/10 flagged ("Linux system administration trainee")
Matching A / B / C / D	54.8 / 52.0 / 21.1 / 21.1	56.5 / 52.0 / 26.1 / 21.1
M2.4 profiles on 320 records (top-1 domain / top-3 / MRR)	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0
Matching ↔ skill-gap findings	2	0
Grounding violations / needs-review items	0 / 4	0 / 0
Fault injections detected	9/9	10/10
Service hand-off chain	all hold	all hold
Conversation turns / full conversations	12/16 / 5/7	15/16 / 6/7
Discovery: requested area retrieved	0/3	3/3

25. Demo Workflow

Uses only real application functionality against a real, persisted database. No scripted responses, seeded fake users or hardcoded metrics are involved. Use a resume that is permitted for demonstration (e.g. a synthetic test resume); do **not** commit it.

Before the demo: follow §28 (index built, backend and frontend running). Leave `LLM_PROVIDER=none` for a fully deterministic, offline demo; results are labelled "Grounded Fallback". For a clean start, point `DATABASE_URL` and `RESUME_STORAGE_DIR` at a fresh location.

Step	Action	What to show
1	Sign up (`/signup`), sign out, sign in.	Validation (e.g. reserved email domains rejected), HttpOnly session.
2	Dashboard / Progress before any resume.	"No resume analyzed yet. No skills extracted yet. No job matches yet."; tracker counts 0 — nothing fabricated.
3	Career Profile — fill education, interests, target roles.	Profile persisted; Progress shows completion computed from saved fields.
4	Resume Analyzer — upload PDF/DOCX, *Process resume*.	Upload → extract → sections → structure pipeline.
5	Resume results.	Extracted skills, education, experience (or "No experience extracted"), projects, summary.
6	Dashboard.	Counts and lists now come from the processed resume.
7	Career Recommendations.	Ranked matches with % match, reasoning, missing required skills. Then search a genuine query (e.g. "python backend developer") and an off-topic query (e.g. "chef pastry baking") to show the low-confidence notice — results are still shown.
8	Job details → *Check my match*.	Deterministic score, matched / missing skills, related-skill partial credit explanation.
9	*Analyze Skill Gaps*.	Readiness %, critical / partial / preferred gaps, quoted resume evidence, recommendations.
10	*Customize Application* → generate.	Keyword alignment (supported / partial / unsupported), tailored summary, reordered skills, ranked projects, cover letter, "Grounding check passed", export buttons.
11	*Prepare for Interview* → generate; practise one question.	Questions tied to the candidate's projects and the job; mock-answer feedback.
12	*Add to Tracker and link*; open the application; set status, application date, deadline, interview date/time, notes; save. Add one manual application with a deadline.	Persisted after reload; summary counts; Upcoming actions (deadline reminder, interview).
13	AI Career Assistant: "Find internships in cybersecurity." → "Compare me with JOB-xxxx." → "What skills am I missing?" → "Which of those should I learn first?"	Area-specific discovery, fit explanation, context retained across follow-ups. Optionally show the CV06 limitation with a salary-negotiation question.

M4.4 manual validation (2026-10-05): all 13 steps were executed in the browser against a fresh scratch SQLite database with `LLM_PROVIDER=none`, using a synthetic DOCX resume (fictional candidate, kept outside the repository). Observed: empty states correct on a fresh account; 6 skills / 1 education / 2 projects extracted; off-topic search showed the notice; skill gap and customization grounded with passing checks; the cover letter kept its job-title opening (E5); tracker data persisted across reload with correct reminders; assistant behaviour matched the evaluation, including the CV06 fallback. The Learning Roadmap page correctly states that roadmap generation is not available.

26. Known Limitations

1. **General LLM-style questions (CV06).** Free-form career questions (e.g. salary negotiation) receive a fixed capability message in deterministic mode. Unresolved; the strict xfail remains.
2. **Matching margins below target.** A−B = 4.5 (target ≥ 10) and C−D = 5.0 (target ≥ 10). Variants that widened the margins (E3a–E3c) caused ordering inversions or M2 profile regressions and were rejected.
3. **Retrieval confidence flag is heuristic.** 11/12 off-topic evaluation queries and 0/26 genuine queries flagged; on held-out queries one genuine query ("Linux system administration trainee") was flagged; "UI UX design internship with Figma" is still not flagged because its words exist in the catalogue. It is a notice only — results are never suppressed.

4. **No live LLM evaluation.** The automated evaluation disables live calls by design. LLM output quality, latency, cost and token usage were not evaluated; the LLM layer is covered by fake-provider unit tests only.
5. **Small evaluation set.** 26 labelled retrieval queries (rule-based labels, not human judgments), 12 off-topic queries, 5 synthetic candidates / 8 pairs, 16 conversation turns. Results describe these fixtures and are not statistically representative of all users, resumes or jobs.
6. **Curated catalogue.** 320 project-authored postings in 15 technology domains; no live job feeds, no real application links, no deadlines in the data. Non-technology fields are out of coverage.
7. **Resume parsing** is rule-based: no OCR (scanned PDFs fail explicitly), and unusual layouts may extract imperfectly.
8. **Learning Roadmap and roadmap progress** are not implemented (the pages say so). Legacy tables `learning_roadmaps`, `roadmap_items`, `selected_jobs` and `progress_events` exist in the schema but have no active feature; they were left in place.
9. **Security scope** is that of a local project (see §22 "Not implemented").
10. **No schema migrations**: `create_all` plus one ad-hoc column helper.
11. **Single-process caches** (M4.3 E10) assume one process; multiple workers would each hold their own copy.
12. **Frontend** has no automated UI tests; one pre-existing ESLint warning remains.

27. Future Enhancements

- Evaluate the LLM path with a live model on a fixed, reviewed prompt set (quality, grounding violations, latency, token usage) and resolve CV06.
- Larger, human-judged retrieval and matching benchmarks; revisit matching margins with more scenarios.
- A trained or calibrated out-of-coverage detector to replace the coverage heuristic.
- Learning-roadmap generation from skill gaps.
- Real job-feed integration with deadlines and application URLs.
- OCR for scanned resumes.
- Production hardening: HTTPS + `secure` cookies, CSRF protection, rate limiting, password reset, migrations (Alembic), structured logging, containerisation and CI.
- Automated frontend tests.

28. Setup / Run Instructions

Prerequisites: Python 3.12, Node.js with npm, Git. Commands below are for Windows PowerShell from the repository root (adapt paths on macOS/Linux).

```
# 1. Backend dependencies
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r backend\requirements.txt

# 2. Optional configuration (defaults work without it)
copy backend\.env.example backend\.env      # keep LLM_PROVIDER=none for offline use
copy .env.example .env                      # VITE_API_BASE_URL, default http://localhost:8000

# 3. Build the FAISS index (gitignored; first run downloads all-MiniLM-L6-v2 once)
cd backend
..\.venv\Scripts\python.exe scripts\validate_job_dataset.py
..\.venv\Scripts\python.exe scripts\build_job_vector_index.py

# 4. Run the API (creates the SQLite database on startup)
..\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
#    http://127.0.0.1:8000/health  -> {"status":"ok"}
#    http://127.0.0.1:8000/docs    -> Swagger UI;  /openapi.json -> schema

# 5. Frontend (new terminal, repository root)
npm install
npm run dev          # http://localhost:5173
```

Verification commands:

```
cd backend; ..\.venv\Scripts\python.exe -m pytest -q
..\.venv\Scripts\python.exe scripts\run_m4_evaluation.py --output-dir $env:TEMP\m4_eval
cd ..; npx tsc -b; npm run lint; npm run build
```

`run_m4_evaluation.py` without `--output-dir` overwrites the tracked `M4_3_OPTIMIZED_EVALUATION.md` / `m4_3_optimized_results.json`; the frozen M4.2 baseline is never overwritten.

Key environment variables (`backend/.env.example`): `APP_ENV`, `DEBUG`, `FRONTEND_URL`, `DATABASE_URL` (relative SQLite paths resolve against `backend/`), `RESUME_STORAGE_DIR`, `EMBEDDING_MODEL_NAME`, `EMBEDDING_ALLOW_DOWNLOAD`, `LLM_PROVIDER` (`none` | `openai_compatible`), `LLM_MODEL`, `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_TIMEOUT_SECONDS`.

29. API Documentation

Interactive documentation is served by FastAPI at **`/docs`** (Swagger UI), with the API schema at **`/openapi.json`** (both verified to return HTTP 200 in M4.4). Groups marked *(session required)* need a valid session cookie: unauthenticated requests receive 401, and resources owned by another user return 404. `GET /health` and the read-only opportunity endpoints (`/api/jobs/*`) are public. In the Auth group, register and login are public, `GET /api/auth/me` returns 401 without a valid session, and logout clears the session if one exists.

Group	Method & path
Health	`GET /health`
Auth	`POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`, `POST /api/auth/logout`
Profile (session required)	`POST /api/profiles`, `GET /api/profiles/{profile_id}`, `PATCH /api/profiles/{profile_id}`
Resume (session required)	`POST /api/profiles/{profile_id}/resumes`, `GET /api/profiles/{profile_id}/resumes`, `GET /api/resumes/{resume_id}`, `POST /api/resumes/{resume_id}/extract-text`, `GET /api/resumes/{resume_id}/extraction`, `POST /api/resumes/{resume_id}/detect-sections`, `GET /api/resumes/{resume_id}/sections`, `POST /api/resumes/{resume_id}/structure`, `GET /api/resumes/{resume_id}/structured`
Candidate context (session required)	`POST /api/profiles/{profile_id}/candidate-context`, `GET /api/profiles/{profile_id}/candidate-context?resume_id=`
Jobs (public, read-only)	`GET /api/jobs/search?q=&top_k=` (1–20), `GET /api/jobs/{job_id}`

Group	Method & path
Matching (session required)	`GET /api/resumes/{resume_id}/job-matches`
Skill gap (session required)	`POST /api/resumes/{resume_id}/skill-gap`, `GET /api/resumes/{resume_id}/skill-gap/{job_id}`
Customization (session required)	`POST /api/resumes/{resume_id}/application-customizations`, `GET …/application-customizations`, `GET …/application-customizations/{customization_id}`, `PATCH …/{customization_id}`, `POST …/{customization_id}/regenerate`, `GET …/{customization_id}/export`
Interview prep (session required)	`POST /api/resumes/{resume_id}/interview-preparations`, `GET …/interview-preparations`, `GET …/{prep_id}`, `POST …/{prep_id}/regenerate`, `POST …/{prep_id}/mock-answer`
Assistant (session required)	`POST /api/career-assistant/conversations`, `GET /api/career-assistant/conversations`, `GET …/conversations/{conversation_id}`, `POST …/conversations/{conversation_id}/messages`, `DELETE …/conversations/{conversation_id}`
Applications (session required)	`POST /api/applications`, `GET /api/applications` (filters/sort), `GET /api/applications/summary`, `GET /api/applications/reminders?days=` (1–60), `GET /api/applications/{application_id}`, `PATCH /api/applications/{application_id}`, `DELETE /api/applications/{application_id}`

Database

SQLite via SQLAlchemy (`backend/app/models/`). Main tables and ownership:

Table	Owner / relation
`users`	account (Argon2id `password_hash`)
`auth_sessions`	→ users; SHA-256 `token_hash`, `expires_at`
`candidate_profiles`	→ users (one per user)
`resumes`	→ candidate_profiles
`resume_extractions`, `resume_sections`, `structured_resumes`	→ resumes
`candidate_contexts`	→ candidate_profiles + resumes
`skill_gaps`	→ users (persisted analyses)
`application_customizations`, `interview_preparations`	→ users + resumes, versioned per job
`conversations` → `conversation_messages`	→ users (active resume optional)
`applications`	→ users; optional → application_customizations, interview_preparations; unique (`user_id`, `job_id`)
`selected_jobs`, `learning_roadmaps`, `roadmap_items`, `progress_events`	legacy / not used by an active feature (kept)

Opportunity data is **not** in the database; it is read from the canonical JSON dataset.

30. Repository / Git Hygiene

Remote `origin` = `https://github.com/jeswinjarald-hash/ai-career-companion.git`; work branch `develop`.

Version-controlled: source (`src/`, `backend/app/`), tests, scripts, the canonical dataset and schema, evaluation cases, the frozen M2.4 / M4.2 results, the M4.3 experiment log and final results, documentation, `.env.example` files, and npm/pip manifests.

Intentionally not tracked (`.gitignore`): `.env` files (real keys), SQLite databases (`backend/data/*.db`), uploaded resumes (`backend/data/resumes/`), the generated FAISS index (`backend/data/vector_store/`), the generated browser test fixture (`backend/data/browser_text_resume.docx`; its generator script is tracked), `node_modules/`, `dist/`, Python caches, virtual environments (`.venv/`, `.venv-*/`), pytest caches, logs, IDE/OS files. `.claude/` (local tool launch configuration) is left untracked.

M4.4 audit (before committing): tracked files contained no `.env`, database, resume, index, cache or build output; a pattern scan of all tracked and to-be-committed files found no API keys, tokens, private keys or credential literals; evaluation case files contain no e-mail addresses; SHA-1 hashes of the dataset, schema, CSV, index files and all evaluation results were identical before and after the M4.4 runs.